

Emilio Corchado · Héctor Quintián ·
Alicia Troncoso Lora · Esteban Jove ·
Pablo García Bringas · Paolo Fosci ·
Francisco Martínez Álvarez (Eds.)

Communications in Computer and Information Science

3046

Soft Computing Models in Industrial and Environmental Applications

21st International Conference, SOCO 2026
Marbella, Spain, June 18–19, 2026
Proceedings



Springer

Communications in Computer and Information Science

3046

Series Editors

Gang Li , *School of Information Technology, Deakin University, Burwood, VIC, Australia*

Joaquim Filipe , *Polytechnic Institute of Setúbal, Setúbal, Portugal*

Zhiwei Xu, *Chinese Academy of Sciences, Beijing, China*

Rationale

The CCIS series is devoted to the publication of proceedings of computer science conferences. Its aim is to efficiently disseminate original research results in informatics in printed and electronic form. While the focus is on publication of peer-reviewed full papers presenting mature work, inclusion of reviewed short papers reporting on work in progress is welcome, too. Besides globally relevant meetings with internationally representative program committees guaranteeing a strict peer-reviewing and paper selection process, conferences run by societies or of high regional or national relevance are also considered for publication.

Topics

The topical scope of CCIS spans the entire spectrum of informatics ranging from foundational topics in the theory of computing to information and communications science and technology and a broad variety of interdisciplinary application fields.

Information for Volume Editors and Authors

Publication in CCIS is free of charge. No royalties are paid, however, we offer registered conference participants temporary free access to the online version of the conference proceedings on SpringerLink (<http://link.springer.com>) by means of an http referrer from the conference website and/or a number of complimentary printed copies, as specified in the official acceptance email of the event.

CCIS proceedings can be published in time for distribution at conferences or as post-proceedings, and delivered in the form of printed books and/or electronically as USBs and/or e-content licenses for accessing proceedings at SpringerLink. Furthermore, CCIS proceedings are included in the CCIS electronic book series hosted in the SpringerLink digital library at <http://link.springer.com/bookseries/7899>. Conferences publishing in CCIS are allowed to use our online conference service (Meteor) for managing the whole proceedings lifecycle (from submission and reviewing to preparing for publication) free of charge.

Publication process

The language of publication is exclusively English. Authors publishing in CCIS have to sign the Springer CCIS copyright transfer form, however, they are free to use their material published in CCIS for substantially changed, more elaborate subsequent publications elsewhere. For the preparation of the camera-ready papers/files, authors have to strictly adhere to the Springer CCIS Authors' Instructions and are strongly encouraged to use the CCIS LaTeX style files or templates.

Abstracting/Indexing

CCIS is abstracted/indexed in DBLP, Google Scholar, EI-Compendex, Mathematical Reviews, SCImago, Scopus. CCIS volumes are also submitted for the inclusion in ISI Proceedings.

How to start

To start the evaluation of your proposal for inclusion in the CCIS series, please send an e-mail to ccis@springer.com

Emilio Corchado · Héctor Quintián ·
Alicia Troncoso Lora · Esteban Jove ·
Pablo García Bringas · Paolo Fosci ·
Francisco Martínez Álvarez
Editors

Soft Computing Models in Industrial and Environmental Applications

21st International Conference, SOCO 2026
Marbella, Spain, June 18–19, 2026
Proceedings

Editors

Emilio Corchado 
University of Salamanca
Salamanca, Spain

Héctor Quintián 
University of A Coruña
Ferrol, La Coruña, Spain

Alicia Troncoso Lora 
Pablo de Olavide University
Seville, Spain

Esteban Jove 
University of A Coruña
Ferrol, La Coruña, Spain

Pablo García Bringas 
University of Deusto
Bilbao, Spain

Paolo Foschi 
University of Bergamo
Bergamo, Italy

Francisco Martínez Álvarez 
Pablo de Olavide University
Seville, Spain

ISSN 1865-0929

ISSN 1865-0937 (electronic)

Communications in Computer and Information Science

ISBN 978-3-032-29253-7

ISBN 978-3-032-29254-4 (eBook)

<https://doi.org/10.1007/978-3-032-29254-4>

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

This volume contains accepted papers presented at the *21st International Conference on Soft Computing Models in Industrial and Environmental Applications* (SOCO 2026), which was held in the beautiful city of Marbella, Spain, in June 2026.

Soft computing represents a collection or set of computational techniques in machine learning, computer science, and some engineering disciplines which investigate, simulate, and analyse very complex issues and phenomena.

After a single-blind peer-review process in which submissions received three reviews each on average, the SOCO 2026 International Program Committee selected 58 papers for publication in these conference proceedings out of 121 submissions, resulting in an acceptance rate of 48%. In this edition, a particular emphasis was put on organising special sessions. Seven special sessions were organised related to relevant topics such as Efficiency and Explainability in Machine Learning and Soft Computing, Time Series Forecasting in Industrial and Environmental Applications, Quantum Computing, EUropean Transnational Educational, Intelligent Techniques Applied to Modelling and Control in Engineering Focused on Marine Energy Systems and Robotics, Machine Learning and Computer Vision in Industry 4.0, and Optimization, Modeling and Control by Soft Computing Techniques

The selection of papers was extremely rigorous to maintain the high quality of the conference. We want to thank the members of the Program Committee for their hard work during the review process. This is a crucial process for creating a high-standard conference; the SOCO conference would not exist without their help.

SOCO 2026 enjoyed outstanding keynote speeches by distinguished guest speakers: Prof. Michal Wozniak at Wroclaw University of Science and Technology (Poland) and Prof. Enrique A. de la Cal Marín at University of Oviedo (Spain).

SOCO 2026 has teamed up with “Neurocomputing” (Elsevier) and “Logic Journal of the IGPL” (Oxford University Press) for a suite of special issues, including selected papers from SOCO 2026.

Particular thanks go as well to the conference’s main sponsors, Startup Olé, BISITE research group at the University of Salamanca, CTC research group at the University of A Coruña, and the University of Salamanca. They jointly contributed actively and constructively to the success of this initiative.

This activity was supported by grant PID2022-137152NB-I00 funded by MICIU/AEI/10.13039/501100011033 and by ERDF/EU.



Motzkin Neighborhood Evaluation in Iterated Local Search and Simulated Annealing for the Permutational Flow Shop Problem

Remigiusz Wojewódzki^(✉) and Wojciech Bożejko

Wrocław University of Science and Technology, Wrocław, Poland
remigiusz.wojewodzki@gmail.com, wojciech.bozejko@pwr.edu.pl

Abstract. In this paper, we investigate the impact of using the *Motzkin* neighborhood in the permutation flow shop problem (PFSP) for two metaheuristic scheduling algorithms: Iterated Local Search and Simulated Annealing. The goal is to minimize $C_{\max}$, understood as the completion time of all jobs. The Motzkin neighborhood enables performing multiple swaps of permutation elements during a single iteration, i.e., simultaneous swaps of multiple permutation elements with allowed nesting, while prohibiting crossings of the lines connecting the elements. The use of dynamic programming reduces the computational complexity and speeds up the algorithm. In this paper, we compare the *Motzkin* neighborhood with the *Fibonacci* neighborhood and the classical *Dynasearch* and *Adjacent* neighborhoods. The experiments were conducted on Taillard benchmark instances for many different time limits. The obtained results allow us to conclude that the choice of neighborhood has a significant impact on the quality of the result obtained by the algorithm within the given time. For all investigated time limits, *Dynasearch* achieved the best results (the smallest average gap), while the *Fibonacci* neighborhood is at the other end of the ranking. The *Motzkin* neighborhood examined in detail in this work is, in most cases, ranked better than *Adjacent*. This advantage becomes clearly visible for larger allowed computation times. The obtained results suggest that neighborhoods using multi-swap moves constitute an interesting alternative to their classical (simpler) counterparts, noting that further computational optimization of complex neighborhoods is required.

Keywords: Flow shop scheduling · Metaheuristics · Iterated Local Search · Simulated annealing · Composite neighborhood · Motzkin

1 Introduction

The Permutation Flow Shop Problem (PFSP), mathematically defined in [2], consists of finding a permutation of n jobs processed on m machines in the same order. The objective is to minimize the completion time of all jobs (makespan)

$C_{\max}$. The problem is known to be NP-hard for $m \geq 3$, which motivates the use of metaheuristic methods. In metaheuristic algorithms, the neighborhood of a solution x is defined as the set of solutions that can be obtained through admissible modifications of x [9].

In addition to the choice of the general algorithm, the selection of the neighborhood structure is equally important. In the classical approach, neighborhood elements perform simple moves, which results in the solution space being explored locally and relatively shallowly. In contrast, composite neighborhoods allow for complex moves within a single iteration, potentially leading to faster convergence and better solution quality. However, this complexity comes at a significant cost in terms of the time required for a single iteration.

Iterated Local Search (ILS) [4, 6] and Simulated Annealing (SA) [5] are among the most widely applied metaheuristic frameworks for combinatorial optimization. This work extends the study reported in [10], where a new type of neighborhood based on Motzkin numbers, called the *Motzkin* neighborhood, was proposed. This neighborhood enables a complex compound move within a single iteration, allowing swaps of any two elements of the permutation as well as swaps corresponding to pairs of indices nested with respect to one another (i.e., nested arcs). Intersections of the corresponding arcs are forbidden.

In the present paper, the focus is placed on an experimental investigation of the behavior and the impact of the neighborhood on the solutions generated by metaheuristic algorithms. For this purpose, we performed experiments for various input sizes and time limits, which allowed us to examine the behavior of the *Motzkin* neighborhood under different conditions. Additionally, a comparative analysis with other neighborhoods—including the classical *Adjacent* swap, the *Dynasearch* neighborhood [3], and the *Fibonacci* neighborhood—was performed in order to identify criteria for selecting an appropriate neighborhood for the problem under study.

2 Problem Definition

Permutation Flow Shop Problem. Given a matrix of processing times $P = [p_{k,j}]_{m \times n}$, where $k = 1, \dots, m$ denotes a machine and $j = 1, \dots, n$ denotes a job, while a permutation π determines the jobs performed in the same order on all machines.

Properties of permutations and the relationships between them can be analyzed using positive definite functions on permutation groups. This provides theoretical foundations for studying dependencies between job sequences [1].

Let $C_{k,j}$ denote the completion time of job j executed in the order π on machine k . The following relationship therefore holds:

$$C_{1,j} = C_{1,j-1} + p_{1,\pi(j)}, \quad C_{k,1} = C_{k-1,1} + p_{k,\pi(1)}, \quad (1)$$

$$C_{k,j} = \max\{C_{k,j-1}, C_{k-1,j}\} + p_{k,\pi(j)}, \quad \text{for } k = 1, \dots, m, j = 1, \dots, n. \quad (2)$$

The value of the objective function equals $C_{\max} = C_{m,n}$, i.e., the completion time of the last job on the last machine. Hence, the PFSP reduces to finding a permutation π that minimizes $C_{\max}$.

Neighborhood Structures. Following [9], a *neighborhood* of a solution x is the set of solutions obtained by an admissible modification of x . It determines how the solution space is explored and exploited during the search. The choice of the neighborhood structure determines the search pattern in the solution space, balancing exploration and exploitation for a given space.

Characteristics of the Motzkin Numbers. The Motzkin numbers M_n , defined in [7, 10], count *planar*, i.e., non-crossing, configurations in a set of n ordered points. Each point may be isolated, be the beginning of an arc, or its end, under the restriction that arcs cannot intersect. In the presented approach, these numbers constitute, in a sense, a generalization of the Catalan numbers, with the reservation that the former allow isolated points.

3 Motzkin Neighborhood Structures

The neighborhood proposed in [10] constitutes a strictly structured neighborhood, in which each swap of two elements is represented by a pair of their corresponding indices (i, j) . A set of such pairs is admissible only if the corresponding arcs (created above the index axis) do not intersect and do not share endpoints. Nesting of arcs and elements that are not part of any arc are admissible. In comparison to classical neighborhoods, such an approach allows searching a larger region of solutions in a single iteration. This structure is illustrated in Fig. 1, where an example of an admissible composite move with nested and disjoint arcs is presented.

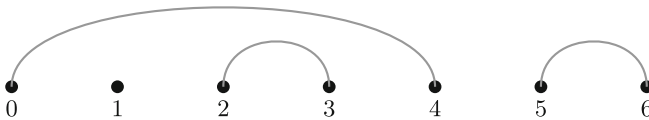


Fig. 1. Example of an admissible composite move: arcs $(0, 4)$, $(2, 3)$, $(5, 6)$.

3.1 Formal Definition

Swapping elements. Given a permutation $\pi = (\pi_0, \pi_1, \dots, \pi_{n-1})$ and a pair of indices (i, j) satisfying $0 \leq i < j < n$, we define the *end swap* as the operation:

$$S_{i,j}(\pi) = (\pi_0, \dots, \pi_{i-1}, \pi_j, \pi_{i+1}, \dots, \pi_{j-1}, \pi_i, \pi_{j+1}, \dots, \pi_{n-1}). \quad (3)$$

This operation swaps the elements indexed by i and j , leaving the intermediate elements $\pi_{i+1}, \dots, \pi_{j-1}$ unchanged. Locally, the segment $[\pi_i, \pi_{i+1}, \dots, \pi_{j-1}, \pi_j]$ is transformed into $[\pi_j, \pi_{i+1}, \dots, \pi_{j-1}, \pi_i]$.

Composite move. We define it as a set of index pairs $\mathcal{M} = \{(i_1, j_1), (i_2, j_2), \dots, (i_k, j_k)\}$. The set $\mathcal{M}$ is *admissible* if it satisfies the following conditions:

1. **Index disjointness:** For any two index pairs $(i_a, j_a), (i_b, j_b) \in \mathcal{M}$ we have $\{i_a, j_a\} \cap \{i_b, j_b\} = \emptyset$ (no shared indices).
2. **No crossings:** The pattern $i_a < i_b < j_a < j_b$ is forbidden for any $(i_a, j_a), (i_b, j_b) \in \mathcal{M}$ (arcs cannot intersect).
3. **Nesting allowed:** The configuration $i_a < i_b < j_b < j_a$ is admissible (the inner pair is entirely contained within the outer pair).

3.2 Algorithm for Constructing a New Permutation

The algorithm determines the optimal set of pairs $\mathcal{M}$ within a single iteration using dynamic programming. After computing the prefix completion columns and evaluating all possible swaps, the algorithm reconstructs the selected set of pairs and applies the determined swaps to construct a permutation π that leads to makespan minimization.

4 Full Enumeration of Motzkin Composite Moves

In this section we analyze the number of admissible composite moves for a given n and relate it to the Motzkin numbers M_n . The first values of the Motzkin numbers for $n = 0, \dots, 10$ are summarized in Table 1, which serve as a reference for the theoretical growth of the neighborhood size.

Table 1. The first Motzkin numbers ($n = 0..10$).

n	0	1	2	3	4	5	6	7	8	9	10
M_n	1	1	2	4	9	21	51	127	323	835	2188

For small values of n , we explicitly list all admissible configurations in order to illustrate the structure of the Motzkin neighborhood.
 $n = 0, 1$. Only the empty configuration exists:

$$\mathcal{M}_1 = \emptyset.$$

$n = 2$. The admissible configurations are:

$$\mathcal{M}_1 = \emptyset, \quad \mathcal{M}_2 = \{(0, 1)\}.$$

$n = 3$. The admissible configurations are:

$$\mathcal{M}_1 = \emptyset, \quad \mathcal{M}_2 = \{(0, 1)\}, \quad \mathcal{M}_3 = \{(0, 2)\}, \quad \mathcal{M}_4 = \{(1, 2)\}.$$

$n = 4$. The set of configurations can be grouped into structural classes:

$$\mathcal{M} \in \{\emptyset, 1 \text{ pair}, 2 \text{ disjoint pairs}, 2 \text{ nested pairs}\}.$$

$n = 5, 6$. The number of configurations increases rapidly following the Motzkin sequence, which makes full enumeration increasingly impractical. Therefore, we restrict the presentation to a structural summary while preserving combinatorial completeness.

To avoid excessive expansion of the manuscript, for $n \geq 7$ we present only the aggregated structure of composite moves. Table 2 presents the distribution of composite moves with respect to the number of pairs.

Table 2. Distribution of composite moves by number of pairs.

n	0 pairs	1 pair	2 pairs	≥ 3 pairs	Total M_n
0	1	0	0	0	1
1	1	0	0	0	1
2	1	1	0	0	2
3	1	3	0	0	4
4	1	6	2	0	9
5	1	10	10	0	21
6	1	15	15	20	51
7	1	21	35	70	127
8	1	28	126	168	323
9	1	36	330	468	835
10	1	45	825	1317	2188

These aggregated values preserve the full combinatorial structure of the Motzkin neighborhood while significantly reducing the exposition length.

Remarks on Growth. The number of admissible composite moves grows exponentially as

$$M_n \sim c \cdot \frac{3^n}{n^{3/2}},$$

which makes full enumeration infeasible for larger n . Nevertheless, dynamic programming allows efficient exploration of the neighborhood in $O(n^3)$ time without explicit generation of all configurations.

The correspondence between composite moves and Motzkin structures provides a combinatorial foundation for the proposed neighborhood and explains its computational behavior.

5 Experimental Setup

The experiment was conducted in the test environment defined in [11], as follows. All experiments were conducted on a MacBook Pro with an M4 Pro processor, 24 GB of RAM, running macOS 26.0.1 (25A362) and Python 3.11.

Benchmarks. Taillard instances [8] of varying (n, m) (e.g., $n \in \{50, 100, 200, 500\}$, $m \in \{10, 20\}$). Each run is constrained by a wall-clock time budget (e.g., 100 ms). Seeds are fixed for reproducibility. For each (n, m) configuration, the reported mean gap values are averaged over the 10 standard Taillard instances. Each instance was executed once with a fixed seed, ensuring full reproducibility.

Parameters. The following parameter values were used in the experiments:

- ILS: $\tau_{\text{tabu}} = 10$
- SA: $T_{\text{init}} = 1000$, $T_{\text{final}} = 1$, $\alpha = 0.95$, reheating factor $r = 1.5$, stagnation threshold 500 (ms).

Metrics. The evaluation considered the best C_{max} obtained over time, convergence curves for different neighborhood strategies, the number of effective moves per second (normalized), and the plateau detection latency. The choice of experimental parameters and metrics was primarily aimed at assessing the impact of different neighborhood structures on the final solution quality rather than evaluating the absolute performance of the algorithms.

6 Results and Discussion

The results presented in Tables 3, 4, 5, 6, 7 and 8 and in the plots of the mean gap as a function of the time limit (Figs. 3 and 4) indicate that the choice of the neighborhood structure has a key impact on the final performance of the algorithm as a function of time. An example of a single run of Simulated Annealing for a representative instance is shown in Fig. 5, illustrating the behavior of the algorithm on a fixed configuration.

The averaged results indicate that, for all analyzed time limits, the ranking of neighborhoods is, in principle, similar. *Dynasearch* has the lowest gap values. It is optimized computationally, which allows for performing a larger number of iterations within the same computation time. Additionally, for large datasets, a single *Dynasearch* iteration takes much more time than the overall time limit. It should be noted that the time-limit condition was checked at the beginning of each algorithm iteration. After the limit was exceeded, it was still possible to finish the already started iteration and include its outcome. Therefore, in some cases the reported results correspond to an effective computation time that exceeds the nominal time limit. This allows us to omit the results for this neighborhood in the further part. In contrast, the *Fibonacci* neighborhood performs the least

Table 3. Average results for $T_{\text{limit}} = 100$ ms.Time limit: $t_{\text{ms}} = 100$

instance	n	m	adj _{ILS}	adj _{SA}	fib _{ILS}	fib _{SA}	dyn _{ILS}	dyn _{SA}	mot _{ILS}	mot _{SA}
tai100_10	100	10	18.41	19.62	17.34	17.34	17.33	17.33	18.98	18.98
tai100_20	100	20	27.55	27.94	24.50	24.50	25.50	25.50	27.20	27.20
tai100_5	100	5	11.84	11.98	11.15	11.15	11.01	11.01	12.48	12.25
tai200_10	200	10	15.40	15.28	14.01	14.01	13.20	13.20	14.79	14.79
tai200_20	200	20	23.26	23.33	21.62	21.62	20.71	20.71	22.06	22.06
tai20_10	20	10	19.59	20.73	29.73	29.73	14.68	14.68	16.61	17.61
tai20_20	20	20	31.89	30.30	36.37	36.37	24.83	24.83	25.56	25.58
tai20_5	20	5	7.18	7.98	17.39	17.39	6.18	6.18	8.39	8.39
tai500_20	500	20	16.25	16.03	15.57	15.57	14.51	14.51	15.93	15.93
tai50_10	50	10	19.65	22.10	20.23	20.23	16.84	17.00	19.86	20.17
tai50_20	50	20	32.54	34.23	32.19	32.23	31.14	31.14	30.57	30.57
tai50_5	50	5	10.71	8.38	12.13	12.13	6.20	6.07	10.35	10.45
AVG			19.52	19.82	21.02	21.02	16.84	16.85	18.57	18.66

Table 4. Average results for $T_{\text{limit}} = 500$ ms.Time limit: $t_{\text{ms}} = 500$

instance	n	m	adj _{ILS}	adj _{SA}	fib _{ILS}	fib _{SA}	dyn _{ILS}	dyn _{SA}	mot _{ILS}	mot _{SA}
tai100_10	100	10	16.92	17.16	17.19	17.19	15.69	16.08	17.22	17.51
tai100_20	100	20	24.98	26.33	23.02	23.02	25.50	25.50	26.30	26.30
tai100_5	100	5	10.89	6.65	11.15	11.15	8.73	8.73	11.99	11.99
tai200_10	200	10	14.10	14.72	12.72	12.72	13.20	13.20	14.79	14.79
tai200_20	200	20	22.09	22.80	19.74	19.76	20.71	20.71	22.06	22.06
tai20_10	20	10	14.91	20.37	29.73	29.73	14.68	14.68	16.61	17.61
tai20_20	20	20	27.35	30.22	36.37	36.37	24.83	24.83	25.49	25.49
tai20_5	20	5	5.23	6.52	17.39	17.39	6.18	6.18	8.39	8.39
tai500_20	500	20	16.09	16.00	15.13	15.13	14.51	14.51	15.93	15.93
tai50_10	50	10	17.97	12.63	20.23	20.23	9.94	9.94	16.77	16.77
tai50_20	50	20	30.51	26.72	32.19	32.19	23.31	23.31	25.70	25.64
tai50_5	50	5	10.22	3.54	12.13	12.13	2.95	2.95	8.31	8.92
AVG			17.61	16.97	20.58	20.58	15.02	15.05	17.46	17.62

favorably over the entire examined range. The *Fibonacci* neighborhood converges rapidly to a local optimum and subsequently stagnates, which explains the identical results observed across different time limits for smaller instances. Incorporating an anti-stagnation mechanism, such as random restarts or a perturbation operator, constitutes a promising direction for future work. At the same time,

Table 5. Average results for $T_{\text{limit}} = 1000$ ms.

 Time limit: $tl_{\text{ms}} = 1000$

instance	n	m	adj _{ILS}	adj _{SA}	fib _{ILS}	fib _{SA}	dyn _{ILS}	dyn _{SA}	mot _{ILS}	mot _{SA}
tai100_10	100	10	16.68	12.14	17.19	17.19	14.03	14.08	16.35	16.42
tai100_20	100	20	24.06	25.41	23.02	23.02	23.43	23.43	24.51	24.51
tai100_5	100	5	10.83	4.43	11.15	11.15	7.04	7.04	11.53	11.53
tai200_10	200	10	13.44	14.36	12.71	12.71	13.20	13.20	14.79	14.79
tai200_20	200	20	21.44	22.49	19.11	19.11	20.71	20.71	22.06	22.06
tai20_10	20	10	13.34	20.25	29.73	29.73	14.68	14.68	16.61	17.61
tai20_20	20	20	25.75	30.22	36.37	36.37	24.83	24.83	25.49	25.49
tai20_5	20	5	4.76	6.31	17.39	17.39	6.18	6.18	8.39	8.39
tai500_20	500	20	15.96	15.93	14.94	14.94	14.51	14.51	15.93	15.93
tai50_10	50	10	17.78	11.81	20.23	20.23	7.46	7.46	16.29	16.35
tai50_20	50	20	29.31	24.92	32.19	32.19	20.25	20.22	23.62	23.96
tai50_5	50	5	9.84	2.44	12.13	12.13	2.95	2.95	8.06	8.52
AVG			16.93	15.89	20.51	20.51	14.11	14.11	16.97	17.13

Table 6. Average results for $T_{\text{limit}} = 2000$ ms.

 Time limit: $tl_{\text{ms}} = 2000$

instance	n	m	adj _{ILS}	adj _{SA}	fib _{ILS}	fib _{SA}	dyn _{ILS}	dyn _{SA}	mot _{ILS}	mot _{SA}
tai100_10	100	10	16.53	9.63	17.19	17.19	11.19	11.10	15.47	15.54
tai100_20	100	20	23.61	19.84	23.02	23.02	21.31	21.31	23.38	23.85
tai100_5	100	5	10.81	3.41	11.15	11.15	4.83	4.78	11.41	11.41
tai200_10	200	10	13.27	13.77	12.71	12.71	13.20	13.20	14.35	14.35
tai200_20	200	20	20.70	21.80	18.65	18.65	20.71	20.71	22.06	22.06
tai20_10	20	10	12.92	19.80	29.73	29.73	14.68	14.68	16.61	17.61
tai20_20	20	20	24.74	30.22	36.37	36.37	24.83	24.83	25.49	25.49
tai20_5	20	5	4.40	6.31	17.39	17.39	6.18	6.18	8.39	8.39
tai500_20	500	20	15.75	15.91	14.48	14.48	14.51	14.51	15.93	15.93
tai50_10	50	10	17.33	10.94	20.23	20.23	6.95	6.95	14.52	16.35
tai50_20	50	20	28.43	24.63	32.19	32.19	16.65	16.69	22.86	23.62
tai50_5	50	5	9.80	1.96	12.13	12.13	2.95	2.95	8.06	8.52
AVG			16.52	14.85	20.44	20.44	13.17	13.16	16.54	16.93

in Figs. 3 and 4 we observe a systematic improvement in solution quality with increasing T_{limit} . The rate of change depends on the applied neighborhood. For some of the applied methods, a rapid transition to the plateau phase is observed, which is visible in Fig. 2. This indicates the occurrence of a saturation effect and limits the possibility of further searching for better solutions.

Table 7. Average results for $T_{\text{limit}} = 5000$ ms.Time limit: $tl_{\text{ms}} = 5000$

instance	n	m	adj _{ILS}	adj _{SA}	fib _{ILS}	fib _{SA}	dyn _{ILS}	dyn _{SA}	mot _{ILS}	mot _{SA}
tai100_10	100	10	16.52	8.31	17.19	17.19	6.48	6.57	14.32	14.82
tai100_20	100	20	23.22	17.52	23.02	23.02	17.62	17.62	21.75	21.92
tai100_5	100	5	10.72	2.40	11.15	11.15	2.34	2.34	9.22	10.73
tai200_10	200	10	13.19	8.51	12.71	12.71	13.20	13.20	14.07	14.26
tai200_20	200	20	19.83	20.18	18.60	18.60	20.71	20.71	20.42	20.85
tai20_10	20	10	12.58	19.58	29.73	29.73	14.68	14.68	16.61	17.61
tai20_20	20	20	22.61	30.16	36.37	36.37	24.83	24.83	25.49	25.49
tai20_5	20	5	4.17	6.24	17.39	17.39	6.18	6.18	8.39	8.39
tai500_20	500	20	15.35	15.82	13.93	13.91	14.51	14.51	15.93	15.93
tai50_10	50	10	17.10	9.71	20.23	20.23	6.95	6.95	13.35	16.35
tai50_20	50	20	27.65	24.44	32.19	32.19	14.80	14.84	20.95	23.44
tai50_5	50	5	9.38	1.96	12.13	12.13	2.95	2.95	8.06	8.52
AVG			16.03	13.74	20.39	20.39	12.10	12.12	15.71	16.53

Table 8. Average results for $T_{\text{limit}} = 10000$ ms.Time limit: $tl_{\text{ms}} = 10000$

instance	n	m	adj _{ILS}	adj _{SA}	fib _{ILS}	fib _{SA}	dyn _{ILS}	dyn _{SA}	mot _{ILS}	mot _{SA}
tai100_10	100	10	16.50	6.19	17.19	17.19	4.11	4.11	13.58	14.40
tai100_20	100	20	22.92	17.06	23.02	23.02	15.50	15.50	19.87	20.69
tai100_5	100	5	10.57	2.37	11.15	11.15	1.93	1.93	8.38	10.73
tai200_10	200	10	13.16	7.23	12.71	12.71	11.66	11.66	14.07	14.07
tai200_20	200	20	19.63	14.61	18.60	18.60	19.32	19.32	20.17	20.17
tai20_10	20	10	12.07	19.58	29.73	29.73	14.68	14.68	16.61	17.61
tai20_20	20	20	22.12	30.16	36.37	36.37	24.83	24.83	25.49	25.49
tai20_5	20	5	4.04	6.24	17.39	17.39	6.18	6.18	8.39	8.39
tai500_20	500	20	14.98	15.53	13.62	13.62	14.51	14.51	15.93	15.93
tai50_10	50	10	17.10	9.06	20.23	20.23	6.95	6.95	13.35	16.35
tai50_20	50	20	26.58	24.32	32.19	32.19	14.73	14.80	20.37	23.44
tai50_5	50	5	9.38	1.81	12.13	12.13	2.95	2.95	8.06	8.52
AVG			15.75	12.85	20.36	20.36	11.45	11.45	15.36	16.32

The *Motzkin* neighborhood is relatively large, but it allows for performing many swaps during a single iteration in a strictly structured manner. On average, this approach outperforms the *Adjacent* neighborhood. This advantage becomes more visible for larger time limits, which suggests an advantage of multi-swap moves requiring sufficiently large time over simpler solutions.

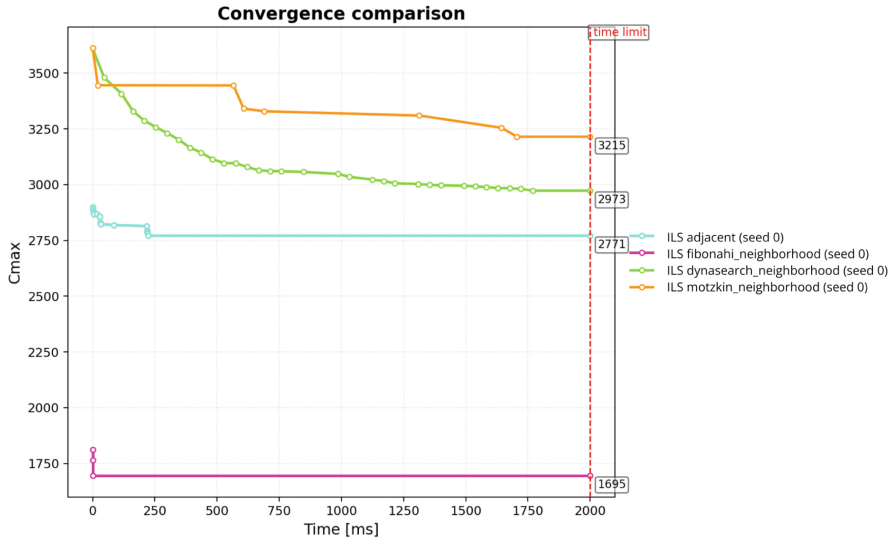


Fig. 2. Iterated Local Search, $t=2000\text{ms}$, $n=200$, $m=10$

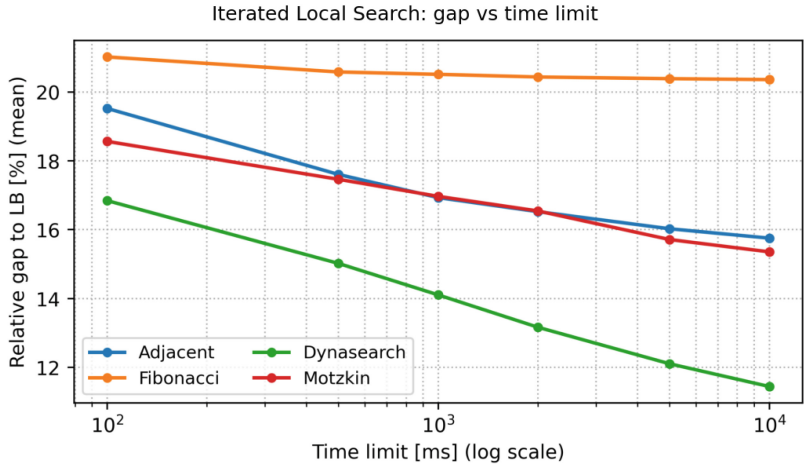


Fig. 3. Comparison of solution quality (mean gap to the lower bound, LB) as a function of the time limit for Iterated Local Search.

To assess the statistical significance of the observed differences, a Wilcoxon signed-rank test was applied to mean gap values across the 12 instance configurations. The superiority of *Dynasearch* over all other neighborhoods is statistically significant across all time limits ($p < 0.01$). The advantage of *Motzkin* over

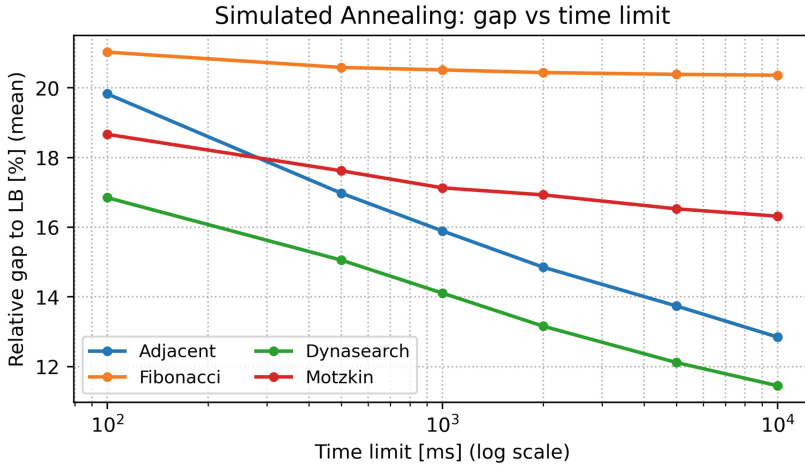


Fig. 4. Comparison of solution quality (mean gap to the lower bound, LB) as a function of the time limit for Simulated Annealing.

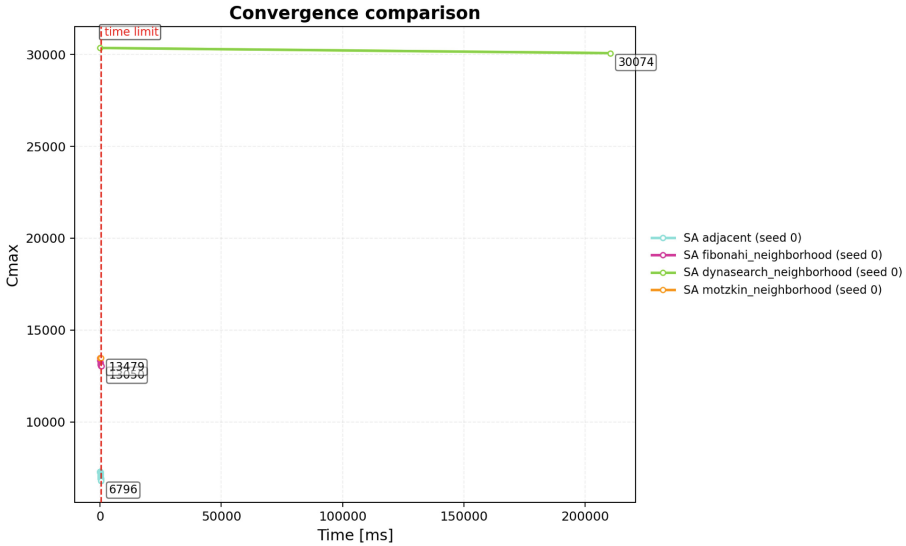


Fig. 5. Simulated Annealing, $t=500\text{ms}$, $n=100$, $m=20$

Fibonacci reaches statistical significance for larger time budgets ($p < 0.05$ for $TL \geq 5000$ ms). The difference between *Motzkin* and *Adjacent* is not statistically significant for most configurations, which is consistent with the limited sample size of 12 instance files. This may also be attributed to the contrasting characteristics of the two neighborhoods: *Adjacent* performs a large number of simple, low-cost

moves per iteration, whereas Motzkin executes fewer but computationally more expensive moves. These competing effects may offset each other in terms of solution quality within a fixed time budget, making it difficult to establish a clear statistical advantage of either approach.

A natural direction for further research is to improve the computational efficiency of the *Motzkin* and *Fibonacci* neighborhood implementations in order to increase the number of iterations within the same time constraint and improve their competitiveness against classical neighborhoods.

References

1. Bożejko, M., Bożejko, W.: Generalized Gaussian processes and relations with random matrices and positive definite functions on permutation groups. arXiv preprint [arXiv:1301.2502](https://arxiv.org/abs/1301.2502) (2013)
2. Bożejko, W., Rudy, J., Wodecki, M.: Parallel neighborhood search for the permutation flow shop scheduling problem. In: Bocewicz, G., Pempera, J., Toporkov, V. (eds.) *Performance Evaluation Models for Distributed Service Networks, Studies in Systems, Decision and Control*, vol. 343, pp. 21–36. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-67063-4_2
3. Campos, V., Glover, F., Laguna, M., Marti, R.: An experimental evaluation of a scatter search for the linear ordering problem. *J. Global Optim.* **21**, 397–414 (2001)
4. Glover, F.: Tabu search—part I. *ORSA J. Comput.* **1**(3), 190–206 (1989)
5. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983)
6. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search. In: Glover, F., Kochenberger, G.A. (eds.) *Handbook of Metaheuristics, International Series in Operations Research & Management Science*, vol. 57, pp. 320–353. Kluwer Academic Publishers (2003)
7. Motzkin, T.S.: Relations between hypersurface cross ratios, and a combinatorial formula for partitions of a polygon. *Bull. Am. Math. Soc.* **54**(4), 352–360 (1948)
8. Taillard, É.: Benchmarks for basic scheduling problems. *Eur. J. Oper. Res.* **64**, 278–285 (1993)
9. Van Laarhoven, P.J.M., Aarts, E.H.L., Lenstra, J.K.: Job shop scheduling by simulated annealing. *Oper. Res.* **40**(1), 113–125 (1992)
10. Wojewódzki, R., Bożejko, W.: Motzkin neighborhood for the permutation flow shop problem. In: Idzikowski, R., Kucharski, M.A. (eds.) *Applications of Advanced Classical and Quantum Computing in Science and Technology: Current Research Developments*, pp. 29–42. Akademicka Oficyna Wydawnicza EXIT, Warszawa (2025)
11. Wojewódzki, R., Bożejko, W.: Hybrid neighborhood evaluation in tabu search and simulated annealing for the permutation flow shop problem. In: *Proceedings of the International Conference on Artificial Intelligence and Soft Computing (ICAISC)* (2026). (Accepted, in press)